

- * System Design
 - Defining the architecture, components and data flow of a system.
 - Blueprint for designing application.

HLD - High Level Design.

- Designing a system at broader, abstract level
- Focus on How the system components
interact with each other

HLD - High level Design.

- Designing a system at broader, abstract level
- Focus on How the system components
Interact with each other

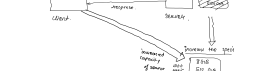
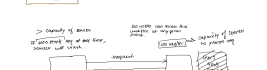
LLD - Low Level Design.

- Designing individual components.
- How each module will be implemented.

LLD

- Low level Design.
- Designing Individual components.
- How each module will be implemented.

A diagram showing two rectangular boxes connected by a double-headed horizontal arrow. The word "ganga" is written above the arrow, and the word "dual" is written below the arrow.



$\therefore$ I will be able to sustain 1000 avg.
 $\therefore$ 11,000 avg. - again increase the capacity.
 Approach: we are increasing the capacity of

Vertical Scaling =

Steps: -

Vertical Scaling

↓

Big Data - Diverse

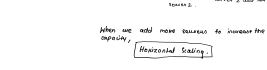
- Limited scalability. [may require an upgrade to a bigger system some day]
- Single point of failure.
- Higher cost.

↓

Horizontal Scaling.

[illegible]

* Horizontal Scalloping
that capacity
in your register.



Traditional

Testing

Results

Modern

Testing

Results



How does Balancer works :

How does balance work :

- ① Removes the key from
- ② Operation which leaves
 $\downarrow$ Based on time

* Load Balancing Algorithms,
To distribute each source third hand
key.
Based in upcoming service.

* Leaf-Boring *Agathidium*
To determine which is
leg.
Based on appearance

★ DNS = Domain Name Service

```

graph LR
    Client[Client] -- "Domain Name" --- DNS[DNS Server]
    DNS --- S1[Server S1]
    DNS --- S2[Server S2]
    DNS --- S3[Server S3]
  
```

* DNS - Domain Name System



Diagram illustrating a DNS lookup process:

- A client sends a query to a DNS server.
- The DNS server checks its cache and finds a record for `www.example.com` with IP `192.168.1.1`.
- The DNS server returns the IP address to the client.



$\Downarrow$
 DNT.

Domain NAMES $\rightarrow$ Address used by
 process in assembly.

$\rightarrow$ Allocation characterizes
 memory address space.

TLD $\rightarrow$ Registry operator $\rightarrow$ Registrar

Top level Domain

- Generic
- Country
- Specific

Soldatman $\rightarrow$ left part

mean, george, car
dog, jungle, count

* Single point of failure

* Single point of failure

```

graph LR
    A[A] --> B[B]
    style B stroke-dasharray: 5 5
  
```



ways to resolve Δ

1. Redundancy.
→ Adding $\rightarrow$

2. Last Balance.

→ Disturb

8. Paulsen Meth.

→

$$\frac{N_{\text{eff}}^{\text{flux}}}{b} \quad \text{---}$$

Root DNS server

⑤

The DNS lookup process.

- ① User may domain name
- ② Check Resolver (computer)

The diagram illustrates the DNS resolution process. A PC sends a request (1) to the Local DNS. The Local DNS checks its cache (2) and if not found, it queries the TLD (3). The TLD refers it to the Authoritative DNS (4). The Authoritative DNS returns the IP (5). The PC then connects to the destination (6). A side note shows that if the Local DNS is a Stub Resolver, it can also query the Stub Resolver (computer's own Resolver) for the IP (e.g., Google's DNS [8.8.8.8]).

Source: [] DNS Server

- it returns it.
- it will go to [] Root DNS - TLD

④ Once IP address is found, it returns to your computer.

Browser cache $\rightarrow$ OS local cache $\rightarrow$ SPONS
 Google DNS
 ↑
 Port
 Domain
 ↑
 TLD